```
export default class FavoriteToggleController extends Controller {
  toggle(): void {
    console.log("Click!")
  }
}
```

Here we removed the connect method and replaced it with a toggle that similarly writes to the console. Now if you reload the page, clicking the new Hide button for favorites will cause the output to be written to the console. This is nice but isn't quite what we're hoping for. What we need is a way to identify the DOM element we want to hide. We can use DOM IDs for that, but Stimulus gives us a shortcut: targets.

We don't show it in our toggle method, but Stimulus does pass one argument to the methods it calls. For TypeScript purposes, this argument is of type CustomEvent, and it allows you access to the target, which is the element that actually hosted the event, and the currentTarget, which is the element that contains the data-action attribute. The two might differ if, say, the click actually happened on an element nested within the element with the data-action. You also have access to normal DOM event methods, such as preventDefault() and stopPropagation().

Stimulus Action Names

Before we get to targets, there are a couple more useful facts about Stimulus actions. First, the event name in the action descriptor can be anything you want, including custom events. Custom events can be a useful way to communicate between controllers. You can also capture global events on the window or document by adding @window or @document, as in ready@document. Typically you'd add these global descriptors to the same DOM element that the controller is defined in.

Second, as with controllers, you can define multiple actions on the same DOM element by adding multiple event descriptors to the same attribute, separated by spaces. This is quite common and is a great way to split up behavior into small controllers and small methods. One nice feature is that if you have multiple responses to the same event, Stimulus guarantees the controller methods will be invoked in the same order the action descriptors are displayed, which allows you to rely on the behavior of the first method already having taken place when you call the second method.